



UNIVERSIDAD INTERNACIONAL DEL ECUADOR – UIDE QUITO
CARRERA DE INGENIERÍA EN SISTEMAS DE LA INFORMACIÓN

TAREA:
APRENDIZAJE AUTÓNOMO 2

ESTUDIANTE:
NICOLE ANGELINA SANI GIORDANO

MATERIA:
LÓGICA DE PROGRAMACIÓN
FECHA DE ENTREGA: 21/06/2026

Índice del documento

I.	OBJETIVOS	1
II.	INTRODUCCIÓN	1
III.	PARTE 1. INICIO DEL DESARROLLO DEL SOFTWARE / CONFIGURACIÓN DEL ENTORNO	2
i.	CONFIGURACIÓN DE UN REPOSITORIO DE GITHUB	2
ii.	DIAGRAMAS DE FLUJO A PARTIR DE LA FUNCIONALIDAD	7
IV.	PARTE 2. DESARROLLO DEL PROGRAMA SELECCIONADO	8
i.	CÓDIGO Y CONTROL DE VERSIONES GITHUB	8
ii.	EJECUCIONES	10
a.	Ejecución sin restricciones	10
b.	Ejecución con restricciones.....	11
V.	CONCLUSIONES DE LA INVESTIGACIÓN	12
VI.	RECOMENDACIONES	12
VII.	ENLACE DEL REPOSITORIO Y EL VIDEO	12
VIII.	REFERENCIAS	13

I. OBJETIVOS

- Desarrollar un programa de software interactivo, utilizando el lenguaje de programación Python aplicando los fundamentos de lógica computacional cómo estructuras de flujo y operadores booleanos.
- Configurar un repositorio en GitHub y enlazarlo con nuestra aplicación de Visual Studio Code para desarrollar un entorno de desarrollo profesional.
- Implementar una solución de software que integre estructuras de control selectivo, operadores lógicos, bucles indeterminados y bucles definidos para desarrollar un programa de mayor nivel de complejidad
- Organizar y documentar el código fuente de manera clara para garantizar su legibilidad y una comprensión sencilla de la lógica implementada.

II. INTRODUCCIÓN

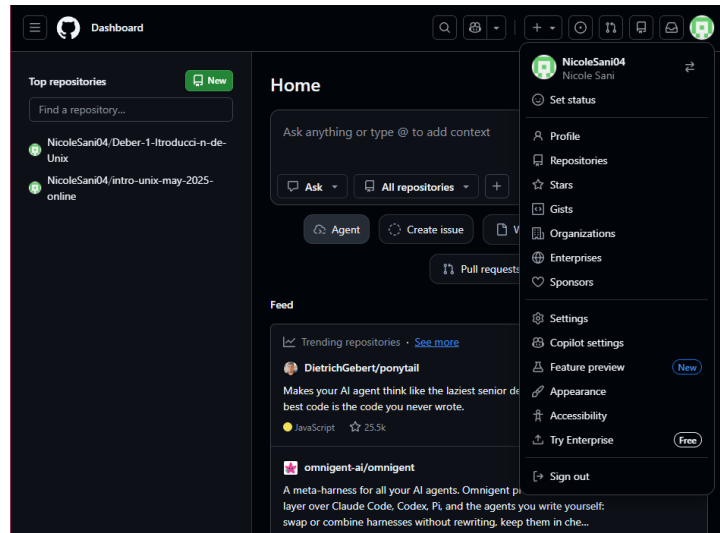
En el presente trabajo vamos a desarrollar un software interactivo en lenguaje Python de nombre” Un Rescate en Mindo, un juego en el cual se aplicarán los aprendizajes obtenidos de los fundamentos de programación, para la resolución de problemas lógicos y estructurados, además se aborda la configuración de un entorno de desarrollo muy común en la industria, utilizando Visual Studio Code y el sistema de control de versiones Git para tener así un respaldo de nuestro código en GitHub.

Se busca desarrollar una arquitectura del programa aplicando rigurosamente estructuras condicionales, el uso de compuertas lógicas (AND, OR, XOR) e implementando estratégicamente estructuras repetitivas cómo son los bucles “while” y “for”, además respaldaremos todo esto aplicando diferentes diagramas de flujo y documentando el código elaborado paso por paso desde la abstracción lógica del problema hasta su correcta ejecución.

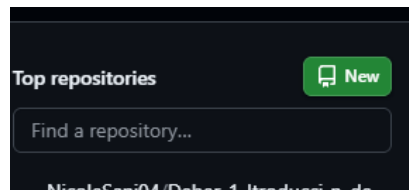
III. PARTE 1. INICIO DEL DESARROLLO DEL SOFTWARE / CONFIGURACIÓN DEL ENTORNO

i. CONFIGURACIÓN DE UN REPOSITORIO DE GITHUB

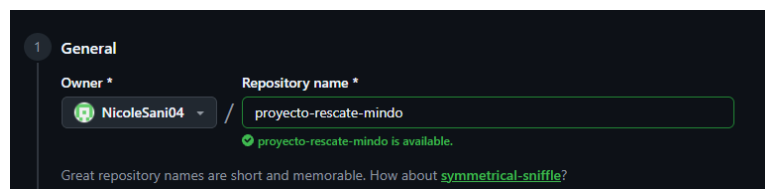
- Primero debemos iniciar sesión en nuestra cuenta de GitHub



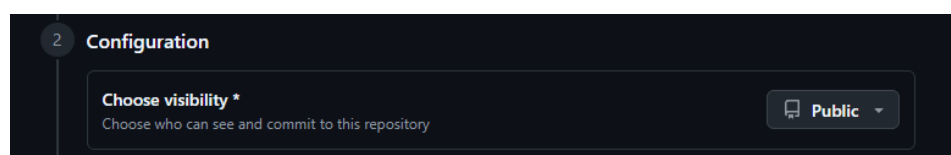
- Debemos crear un nuevo repositorio



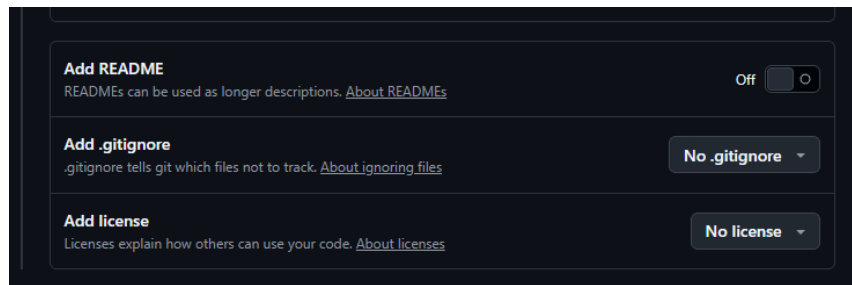
- En el campo de “Repository name” debemos escribir el nombre de nuestro proyecto, que sea claro y sin espacios, en este caso nombraremos nuestro repositorio “proyecto-rescate-mindo”



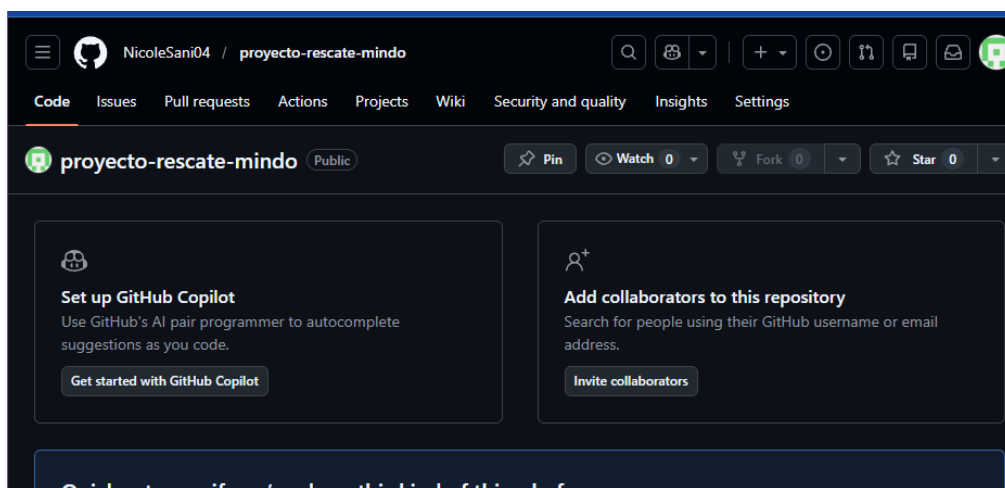
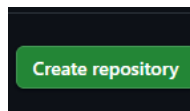
- Debemos definir la visibilidad de nuestro repositorio cómo público en caso de que queramos que los demás puedan revisarlo sin problemas



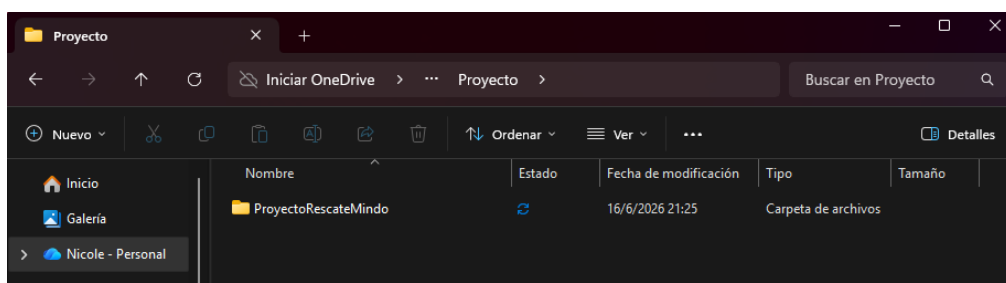
- **NO** marcar las casillas que dicen “Add a README file”, ni “Add .gitignore” ni license, ya que se necesita que el repositorio esté completamente vacío para poderlo conectar a nuestro código de Visual Studio Code sin problemas



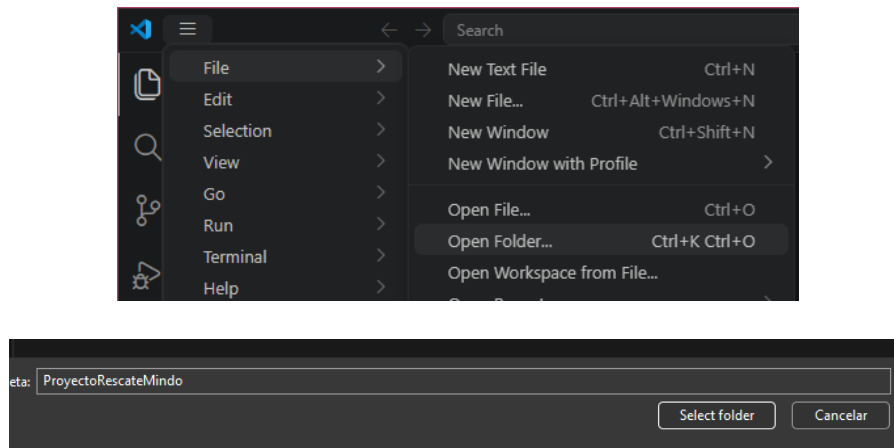
- Hacemos clic al botón “Create repository” para terminar de crear el repositorio



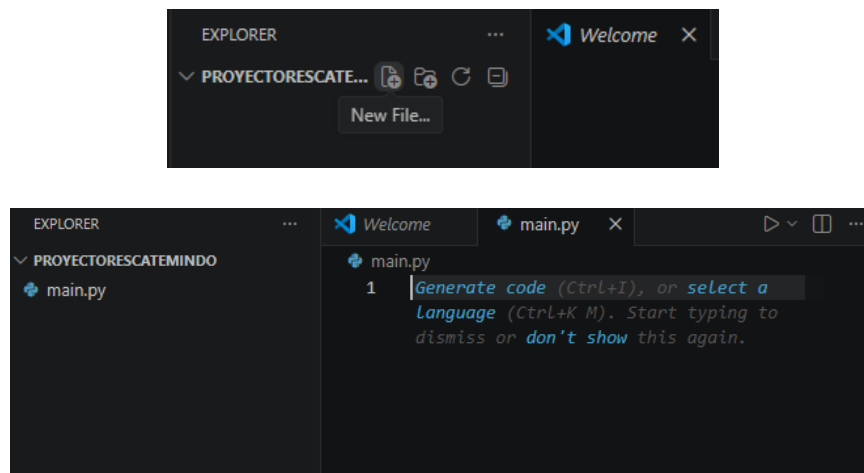
- Vamos a nuestro explorador de archivos y buscamos la ubicación dónde queramos crear y guardar la carpeta de nuestro proyecto “ProyectoRescateMindo”



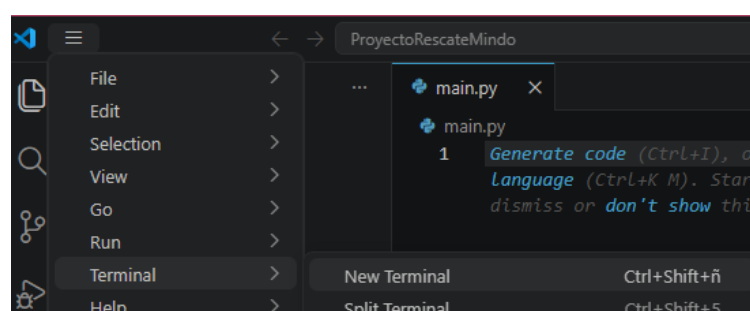
- Abrimos nuestra aplicación de Visual Studio Code y vamos al menú superior y seleccionamos la sección de “File” y le damos clic a la opción “Open Folder” en la cuál seleccionaremos la carpeta en dónde guardaremos todo lo de nuestro programa



- Creamos un archivo con nombre “main.py” dándole clic a la opción de “new file” que aparece sobre el nombre de nuestra carpeta en VS Code



- Abrimos la terminal de nuestro VS Code yendo al menú superior de opciones y seleccionando la sección de “Terminal” y seleccionamos la opción “New Terminal”



- Ejecutamos los siguientes comandos de vinculación en la terminal
 - git init (Para inicializar Git y convertir nuestra carpeta en un repositorio de control de versiones local)

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git init
Initialized empty Git repository in C:/Users/nicol/OneDrive/Documentos/UIDE/Semestre 1/Lógica de Programación/Proyecto/ProyectoRescateMindo/.git/
```

- git add . (Para indicarle a Git que prepare todos los archivos de esa carpeta para ser guardados)

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git add .
```

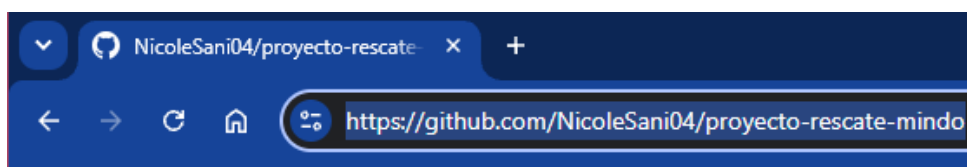
- git commit -m (Para crear un punto de guardado en el que se empaquetaran nuestros cambios con un mensaje explicativo)

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git commit -m "Commit inicial: Creación del código base del juego Rescate en Mindo"
[master (root-commit) 6dfddea] Commit inicial: Creación del código base del juego Rescate en Mindo
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 main.py
```

- git branch -M main (Para definir la rama principal)

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git branch -M main
```

- En este comando debemos copiar el url de nuestro repositorio de GitHub y con esto nuestra carpeta estaría conectada **permanentemente** a GitHub
- git remote add origin «url» (Para vincular nuestra carpeta local con el repositorio)

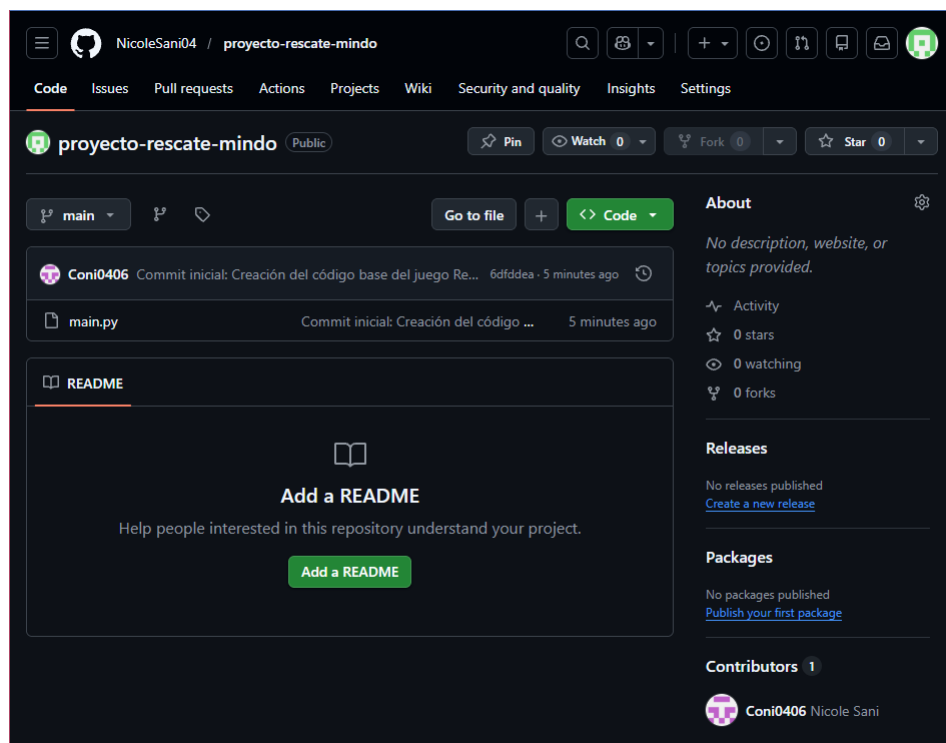


```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git remote add origin https://github.com/NicoleSani04/proyecto-rescate-mindo
```

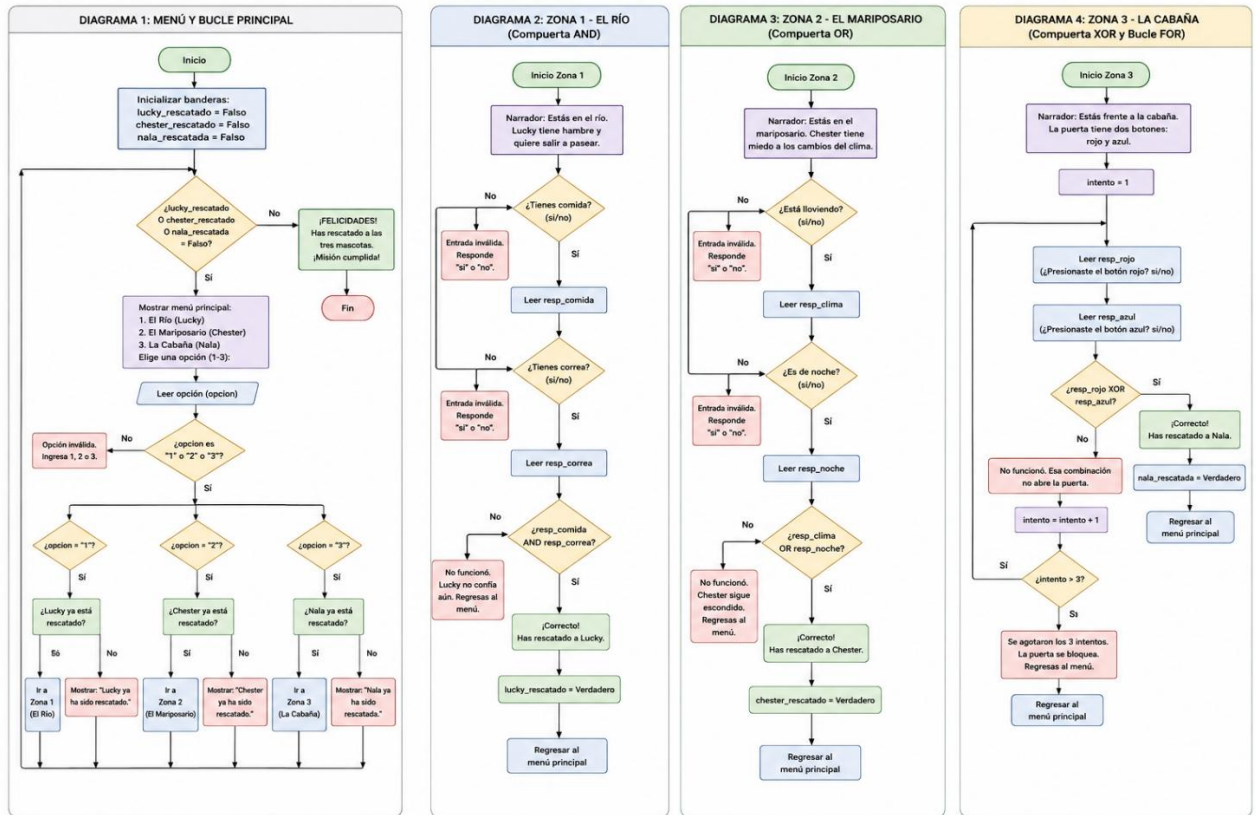
- `git push -u origin main` (Para enviar finalmente nuestros archivos desde nuestra computadora hacia los servidores de GitHub)

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\ProyectoRescateMindo> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 254 bytes | 254.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NicoleSani04/proyecto-rescate-mindo
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- Nuestro repositorio en Git al recargar la página debería actualizarse con los cambios que realizamos en VS Code



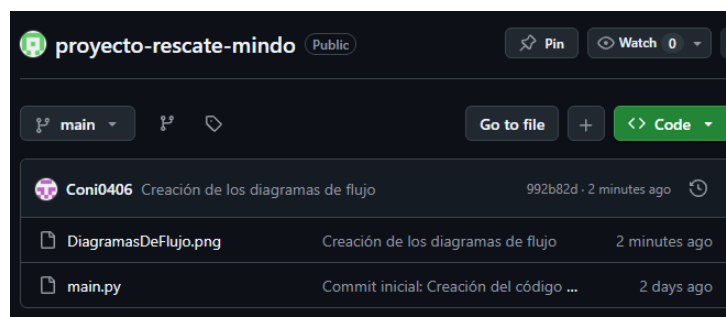
ii. DIAGRAMAS DE FLUJO A PARTIR DE LA FUNCIONALIDAD



- Para subir las imágenes al repositorio que hemos creado, utilizaremos algunos de los comandos antes explicados, no todos ya que la carpeta ya ha sido clonada al repositorio, solo necesitamos guardar los cambios que hemos realizado

```

PS C:\Users\nicol\OneDrive\Documents\UIDE\Semestre 1\Lógica de Programación\Proyecto\Part e 1\proyecto-rescate-mindo> git add .
PS C:\Users\nicol\OneDrive\Documents\UIDE\Semestre 1\Lógica de Programación\Proyecto\Part e 1\proyecto-rescate-mindo> git commit -m "Creación de los diagramas de flujo"
[main 992b82d] Creación de los diagramas de flujo
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 DiagramasDeFlujo.png
PS C:\Users\nicol\OneDrive\Documents\UIDE\Semestre 1\Lógica de Programación\Proyecto\Part e 1\proyecto-rescate-mindo> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.20 MiB | 779.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NicoleSani04/proyecto-rescate-mindo.git
6dfdea..992b82d main -> main
  
```



IV. PARTE 2. DESARROLLO DEL PROGRAMA SELECCIONADO

i. CÓDIGO Y CONTROL DE VERSIONES GITHUB

- Después de realizar los diagramas de flujo, entender la organización y las variables que tendremos en nuestro programa, podemos iniciar el proceso de codificación

```
DiagramasDeFlujo.png X main.py
main.py > resp_clima
1 # -----
2 # PROYECTO: RESCATE EN MINDO
3 # OBJETIVO: Rescatar a Lucky, Chester y Nala usando lógica booleana.
4 # -----
5
6 print("      ¡BIENVENIDO A RESCATE EN MINDO!      ")
7
8 # 1. INICIALIZACIÓN (Estado inicial: Ninguno rescatado)
9 lucky_rescatado = False
10 chester_rescatado = False
11 nala_rescatada = False
12
13 # 2. BUCLE PRINCIPAL (Estructura While)
14 # Se repite MIENTRAS falte al menos una mascota por rescatar
15 while not lucky_rescatado or not chester_rescatado or not nala_rescatada:
16
17     # Muestra opciones disponibles
18     print("\n--- MENÚ PRINCIPAL ---")
19     print("1. El Río (Lucky)")
20     print("2. El Mariposario (Chester)")
21     print("3. La Cabaña (Nala)")
22
23     # 3. VALIDACIÓN DE ENTRADA
24     # El usuario debe ingresar estrictamente 1, 2 o 3
25     opcion = ""
26     while opcion not in ["1", "2", "3"]:
27         opcion = input("Elige una opción para dirigirte (1-3): ").strip()
28         if opcion not in ["1", "2", "3"]:
29             print("Opción inválida. Ingrese 1, 2 o 3.")
30
31     # ZONA 1: EL RÍO (Compuerta AND)
32     if opcion == "1":
33         if lucky_rescatado:
34             print("\nLucky ya ha sido rescatado, escoge otro lugar.")
35         else:
36             print("\nZONA 1: EL RÍO")
37             print("Narrador: Estás en el río, Lucky tiene hambre y necesita una correa.")
38
39             # Validación de respuestas (solo acepta "si" o "no")
40             resp_comida = ""
41             while resp_comida not in ["si", "no"]:
42                 resp_comida = input("¿Tienes comida? (si/no): ").lower().strip()
43                 if resp_comida not in ["si", "no"]:
44                     print("Entrada inválida. Responde 'si' o 'no'.")
45
46             resp_correa = ""
47             while resp_correa not in ["si", "no"]:
48                 resp_correa = input("¿Tienes correa? (si/no): ").lower().strip()
```

```

48     resp_correa = input("¿Tienes correa? (si/no): ").lower().strip()
49     if resp_correa not in ["si", "no"]:
50         print("Entrada inválida. Responde 'si' o 'no'.")
51
52     # LÓGICA AND: Exige que AMBAS condiciones sean verdaderas
53     if resp_comida == "si" and resp_correa == "si":
54         print("¡Correcto! Has rescatado a Lucky.")
55         lucky_rescatado = True
56     else:
57         print("No funcionó. Lucky no confía en tí aún. Regresas al menú.")
58
59 # ZONA 2: EL MARIPOSARIO (Compuerta OR)
60 elif opcion == "2":
61     if chester_rescatado:
62         print("\nChester ya ha sido rescatado.")
63     else:
64         print("\nZONA 2: EL MARIPOSARIO")
65         print("Narrador: Estás en el mariposario. Chester solo saldrá si está lloviendo o es de noche (o las dos juntas).")
66
67         # Validación de respuestas (solo acepta "si" o "no")
68         resp_clima = ""
69         while resp_clima not in ["si", "no"]:
70             resp_clima = input("¿Está lloviendo? (si/no): ").lower().strip()
71             if resp_clima not in ["si", "no"]:
72                 print("Entrada inválida. Responde 'si' o 'no'.")
73
74         resp_noche = ""
75         while resp_noche not in ["si", "no"]:
76             resp_noche = input("¿Es de noche? (si/no): ").lower().strip()
77             if resp_noche not in ["si", "no"]:
78                 print("Entrada inválida. Responde 'si' o 'no'.")
79
80         # LÓGICA OR: Basta con que UNA condición sea verdadera
81         if resp_clima == "si" or resp_noche == "si":
82             print("¡Correcto! Has rescatado a Chester.")
83             chester_rescatado = True
84         else:
85             print("No funcionó. Chester sigue escondido. Regresas al menú.")
86
87 # ZONA 3: LA CABAÑA (Bucle FOR y Compuerta XOR)
88 elif opcion == "3":
89     if nala_rescatada:
90         print("\nNala ya ha sido rescatada.")
91     else:
92         print("\nZONA 3: LA CABAÑA")
93         print("Narrador: Estás frente a la cabaña, la puerta tiene 2 botones: rojo y azul.")
94
95     # 4. BUCLE FOR (Estructura determinada)

```

```

94
95     # 4. BUCLE FOR (Estructura determinada)
96     # Otorga exactamente 3 intentos numerados de forma secuencial (1, 2, 3)
97     for intento in range(1, 4):
98         print(f"\n--- Intento {intento} ---")
99
100         # Validación de respuestas de los botones
101         resp_rojo = ""
102         while resp_rojo not in ["si", "no"]:
103             resp_rojo = input("¿Presionaste el botón rojo? (si/no): ").lower().strip()
104             if resp_rojo not in ["si", "no"]:
105                 print("Entrada inválida. Responde 'si' o 'no'.")
106
107         resp_azul = ""
108         while resp_azul not in ["si", "no"]:
109             resp_azul = input("¿Presionaste el botón azul? (si/no): ").lower().strip()
110             if resp_azul not in ["si", "no"]:
111                 print("Entrada inválida. Responde 'si' o 'no'.")
112
113         # Convertir texto a booleano (True/False) para evaluar el XOR
114         boton_rojo = (resp_rojo == "si")
115         boton_azul = (resp_azul == "si")
116
117         # LÓGICA XOR (^): Exige que EXCLUSIVAMENTE un botón esté activo
118         if boton_rojo ^ boton_azul:
119             print("¡Correcto! Has rescatado a Nala.")
120             nala_rescatada = True
121             break # Rompe el bucle For porque el jugador ya acertó
122         else:
123             print("No funcionó. Esa combinación no abre la puerta.")
124
125         # Verifica si es el último intento para mostrar el bloqueo
126         if intento == 3:
127             print("Se agotaron los 3 intentos. La puerta se bloquea. Regresas al menú.")
128
129 # 5. FIN DEL JUEGO
130 # Se ejecuta solo cuando el bucle While termina (todas las banderas son True)
131 print("¡FELICIDADES! Has rescatado a las tres mascotas.")
132 print("¡Misión cumplida!")

```

- Para finalizar subimos los cambios a nuestro repositorio y habríamos terminado

```
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\Parte 1\proyecto-rescate-mindo> git add .
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\Parte 1\proyecto-rescate-mindo> git commit -m "Codigo concluido"
[main 1194442] Codigo concluido
1 file changed, 132 insertions(+)
PS C:\Users\nicol\OneDrive\Documentos\UIDE\Semestre 1\Lógica de Programación\Proyecto\Parte 1\proyecto-rescate-mindo> git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 2.04 KiB | 2.04 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NicoleSani04/proyecto-rescate-mindo.git
992b82d..1194442 main -> main
```

ii. EJECUCIONES

a. Ejecución sin restricciones

```
¡BIENVENIDO A RESCATE EN MINDO!

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 2

ZONA 2: EL MARIPOSARIO
Narrador: Estás en el mariposario. Chester solo saldrá si está lloviendo o es de noche (o las dos juntas).
¿Está lloviendo? (sí/no): no
¿Es de noche? (sí/no): sí
¡Correcto! Has rescatado a Chester.

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 1

ZONA 1: EL RÍO
Narrador: Estás en el río, Lucky tiene hambre y necesita una correa.
¿Tienes comida? (sí/no): sí
¿Tienes correa? (sí/no): sí
¡Correcto! Has rescatado a Lucky.

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 3

ZONA 3: LA CABAÑA
Narrador: Estás frente a la cabaña, la puerta tiene 2 botones: rojo y azul.

--- Intento 1 ---
¿Presionaste el botón rojo? (sí/no): no
¿Presionaste el botón azul? (sí/no): sí
¡Correcto! Has rescatado a Nala.
¡FELICIDADES! Has rescatado a las tres mascotas.
¡Misión cumplida!
```

b. Ejecución con restricciones

```
--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 1

ZONA 1: EL RÍO
Narrador: Estás en el río, Lucky tiene hambre y necesita una correa.
¿Tienes comida? (si/no): mm
Entrada inválida. Responde 'si' o 'no'.
¿Tienes comida? (si/no): NO
¿Tienes correa? (si/no): si
No funcionó. Lucky no confía en tí aún. Regresas al menú.

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): █
```

```
--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 2

ZONA 2: EL MARIPOSARIO
Narrador: Estás en el mariposario. Chester solo saldrá si está lloviendo o es de noche (o las dos juntas).
¿Está lloviendo? (si/no): no
¿Es de noche? (si/no): no
No funcionó. Chester sigue escondido. Regresas al menú.

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): █
```

```
--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): 8
Opción inválida. Ingresa 1, 2 o 3.
```

```
ZONA 3: LA CABAÑA
Narrador: Estás frente a la cabaña, la puerta tiene 2 botones: rojo y azul.

--- Intento 1 ---
¿Presionaste el botón rojo? (si/no): si
¿Presionaste el botón azul? (si/no): si
No funcionó. Esa combinación no abre la puerta.

--- Intento 2 ---
¿Presionaste el botón rojo? (si/no): no
¿Presionaste el botón azul? (si/no): no
No funcionó. Esa combinación no abre la puerta.

--- Intento 3 ---
¿Presionaste el botón rojo? (si/no): si
¿Presionaste el botón azul? (si/no): si
No funcionó. Esa combinación no abre la puerta.
Se agotaron los 3 intentos. La puerta se bloquea. Regresas al menú.

--- MENÚ PRINCIPAL ---
1. El Río (Lucky)
2. El Mariposario (Chester)
3. La Cabaña (Nala)
Elige una opción para dirigirte (1-3): █
```

V. CONCLUSIONES DE LA INVESTIGACIÓN

- El desarrollo de esta práctica nos demostró que la programación busca trascender la simple escritura de un código, hay muchos procesos que se hacen antes, durante y después, se requiere una correcta abstracción del problema y la aplicación de compuertas lógicas para trasladar las reglas de juego a expresiones booleanas, demostrando así cómo la lógica matemática es la base de la toma de decisiones en nuestro software.
- Se comprobó lo importante de elegir correctamente la estructura de repetición adecuada de acuerdo al escenario (contexto del algoritmo) o lo que se busca resolver, en este caso el bucle “while” busca mantener la ejecución hasta cumplir una condición global, mientras que por otro lado el bucle “for” busca facilitarnos el control de un proceso con límites, todo esto buscando optimizar lo mejor posible el flujo de nuestro programa.
- La configuración de un entorno de trabajo con Visual Studio Code junto a el control de versiones de GitHub busca demostrar una transición entre la programación académica y una práctica real de desarrollo.

VI. RECOMENDACIONES

- Se recomienda mantener cómo práctica regular el uso de diagramas de flujo antes de iniciar la fase de codificación de cualquier programa, graficar nuestro algoritmo nos ayuda a planificar la implementación de las diferentes condiciones anidadas, además, de ayudarnos a prevenir errores lógicos desde nuestra etapa de diseño.
- Se recomienda que, aunque el desarrollo actual de programa es plenamente por consola, se puede proyectar la evolución del juego a un entorno visual más llamativo y entretenido, utilizando librerías de Python como “Tkinter” o “Pygame”, permitiendo así convertir las entradas de texto a botones interactivos para mejorar así la experiencia del usuario.

VII. ENLACE DEL REPOSITORIO Y EL VIDEO

- <https://github.com/NicoleSani04/proyecto-rescate-mindo>
- <https://youtu.be/Fmss1uqZJjs>

VIII. REFERENCIAS

- [1] *Bucle while en Python.* (s/f). El Libro De Python. Recuperado el 17 de junio de 2026, de <https://ellibrodepython.com/while-python>
- [2] *Bucle for.* (s/f). El Libro De Python. Recuperado el 17 de junio de 2026, de <https://ellibrodepython.com/for-python>
- [3] *Source control in VS code.* (s/f). Visualstudio.com. Recuperado el 17 de junio de 2026, de <https://code.visualstudio.com/docs/sourcecontrol/overview>